

Low-Quality Training Data Only?

A Robust Framework for Detecting Encrypted Malicious Network Traffic

1. Introduction

In today's digital world, network security has become very important because cyber attacks are increasing rapidly. Traditional Intrusion Detection Systems (IDS) often face problems such as noisy labels, imbalanced datasets, and poor generalization on unseen attacks. Due to these issues, machine learning models may misclassify malicious traffic and reduce overall detection performance.

To solve these problems, we implemented the RePAIR (Robust and Privacy-Aware Intrusion Recognition) approach for network intrusion detection. The system uses deep learning and data augmentation techniques to improve attack detection accuracy and robustness. The project focuses on handling noisy labels, improving minority attack detection, and enhancing overall model generalization.

2. Objective

The main objective of this project is to develop an intelligent intrusion detection system capable of accurately classifying malicious and benign network traffic even in the presence of noisy and imbalanced data.

The project aims to:

- Improve intrusion detection performance using deep learning techniques.
- Handle noisy labels present in cybersecurity datasets.
- Increase minority attack samples using augmentation methods.
- Improve robustness and generalization of the model.
- Achieve high recall, precision, F1-score, and accuracy for attack detection.

3. Dataset Used

We used the Network Intrusion Dataset from Kaggle based on CICIDS traffic data.

The dataset contains real network traffic captured in `.pcap` files and converted into flow-based features.

Some files used in dataset:

- Monday-WorkingHours
- Tuesday-WorkingHours
- Wednesday-WorkingHours

The dataset contains:

- Benign traffic
- DDoS attacks
- DoS attacks
- PortScan
- Brute Force
- Bot attacks
- Web attacks
- Infiltration attacks

Each row contains multiple network flow features and one label representing the traffic type.

4. Methodology

4.1 Data Preprocessing

In the preprocessing stage, the dataset is cleaned and prepared before model training.

The following preprocessing steps were applied:

- Missing values were replaced with 0.
- Infinite values were converted and handled properly.
- Duplicate and corrupted entries were removed.
- Features were normalized using MinMaxScaler.
- Labels were encoded into numerical format.

This preprocessing improves training stability and model performance.

4.2 Feature Engineering

Important network flow features were extracted from the dataset to represent traffic behavior effectively.

Examples of features:

- Flow Duration
- Packet Length
- Flow Bytes/s
- Flow Packets/s
- Source Port
- Destination Port
- Idle Time
- Active Time

These features help the model differentiate between benign and malicious traffic.

4.3 Autoencoder-Based Feature Learning

An Autoencoder was used for deep feature representation learning.

The Autoencoder:

- Compresses high-dimensional traffic data into compact latent features.
- Removes redundant and noisy information.
- Learns important traffic behavior patterns automatically.

This improves feature quality and helps the classifier perform better.

4.4 Label Noise Handling using RePAIR

The RePAIR framework was used to handle noisy labels present in the intrusion dataset.

In cybersecurity datasets, some traffic samples may contain incorrect labels because:

- Attacks may resemble normal traffic.
- Dataset generation may contain annotation errors.
- Minority attack classes are difficult to classify.

RePAIR improves data quality by:

- Detecting suspicious or incorrectly labeled samples.
- Correcting noisy labels during training.
- Improving robustness against mislabeled traffic.

This helps the model learn more accurate attack patterns.

4.5 GAN-Based Data Augmentation

The dataset was highly imbalanced because some attack classes had fewer samples.

To solve this problem, GAN (Generative Adversarial Network) augmentation was applied.

GAN helps by:

- Generating synthetic minority attack samples.
- Increasing diversity of attack traffic.
- Improving minority class learning.

This prevents the model from becoming biased toward majority classes.

4.6 Ensemble-Based Intrusion Detection

Multiple deep learning models were combined using ensemble learning techniques.

The ensemble approach:

- Combines predictions from multiple classifiers.
- Reduces individual model errors.
- Improves overall detection accuracy and robustness.

This produces more stable and reliable intrusion detection results.

5. Model Pipeline

Data Collection
Data Preprocessing
Feature Engineering
Autoencoder-Based Feature Learning
Label Noise Handling using RePAIR
GAN-Based Data Augmentation
Ensemble-Based Intrusion Detection
Final Evaluation

6. Output and Results

```
C:\Users\jainc\OneDrive\Desktop\transferability-main>python main.py

***** Communication Round 1 *****
.....Device 1.....
100%| 885/885 [06:15<00:00, 2.36it/s]
100%| 111/111 [00:14<00:00, 7.61it/s]
100%| 885/885 [13:57<00:00, 1.06it/s]
100%| 111/111 [00:10<00:00, 10.31it/s]
100%| 111/111 [00:14<00:00, 7.57it/s]
.....Device 2.....
100%| 885/885 [04:38<00:00, 3.18it/s]
100%| 111/111 [00:12<00:00, 8.54it/s]
100%| 885/885 [30:45<00:00, 2.09s/it]
100%| 111/111 [00:13<00:00, 8.41it/s]
100%| 111/111 [00:13<00:00, 8.00it/s]
.....Device 3.....
100%| 885/885 [05:25<00:00, 2.72it/s]
100%| 111/111 [00:13<00:00, 8.10it/s]
4%| 5%| 5%| 5%| 5%|
5%| 5%| 5%| 5%| 5%|
6%| 100%| 885/885 [13:34<00:00, 1.09it/s]
111/111 [00:11<00:00, 10.07it/s]
111/111 [00:11<00:00, 9.58it/s]
.....Device 4.....
100%| 885/885 [04:28<00:00, 3.30it/s]
100%| 111/111 [00:11<00:00, 9.94it/s]
100%| 885/885 [13:59<00:00, 1.05it/s]
100%| 111/111 [00:11<00:00, 9.25it/s]
100%| 111/111 [00:12<00:00, 9.01it/s]
```

```
100%| 111/111 [00:12<00:00, 9.01it/s]
.....Device 5.....
100%| 885/885 [05:44<00:00, 2.57it/s]
100%| 111/111 [00:11<00:00, 9.83it/s]
100%| 885/885 [13:27<00:00, 1.10it/s]
100%| 111/111 [00:12<00:00, 8.85it/s]
100%| 111/111 [00:12<00:00, 8.94it/s]
Device Accuracies: [0.94613645 0.89905842 0.94928368 0.93454873 0.88599187]
Average Accuracy: 0.9230038320118137

***** Communication Round 2 *****
.....Device 1.....
100%| 885/885 [35:18<00:00, 2.39s/it]
100%| 111/111 [00:11<00:00, 9.95it/s]
100%| 885/885 [36:44<00:00, 2.49s/it]
100%| 111/111 [00:12<00:00, 8.95it/s]
100%| 111/111 [00:12<00:00, 8.75it/s]
.....Device 2.....
100%| 885/885 [47:48<00:00, 3.24s/it]
100%| 111/111 [00:15<00:00, 7.08it/s]
100%| 885/885 [49:02<00:00, 3.33s/it]
100%| 111/111 [00:15<00:00, 7.01it/s]
100%| 111/111 [00:15<00:00, 6.96it/s]
.....Device 3.....
100%| 885/885 [44:01<00:00, 2.98s/it]
100%| 111/111 [00:10<00:00, 10.19it/s]
100%| 885/885 [39:04<00:00, 2.65s/it]
111/111 [00:17<00:00, 6.25it/s]
111/111 [00:17<00:00, 6.21it/s]
.....Device 4.....
100%| 885/885 [1:36:01<00:00, 6.51s/it]
100%| 111/111 [00:19<00:00, 5.86it/s]
100%| 885/885 [1:00:20<00:00, 4.63s/it]
100%| 111/111 [00:15<00:00, 7.31it/s]
100%| 111/111 [00:15<00:00, 7.28it/s]
```

```

.....Device 5.....
100%| 885/885 [37:07<00:00, 2.52s/it]
100%| 111/111 [00:13<00:00, 8.45it/s]
100%| 885/885 [37:27<00:00, 2.54s/it]
100%| 111/111 [00:14<00:00, 7.46it/s]
100%| 111/111 [00:15<00:00, 7.38it/s]
Device Accuracies: [0.94629545 0.89918208 0.95656168 0.94726801 0.88560325]
Average Accuracy: 0.9269820918343662

```

Observations

- Accuracy increase after each round
- Model become more better and generalized
- Devices learn from each other without sharing data

Accuracy after first round - Device Accuracies: [0.94613645 0.89905842 0.94928368 0.93454873 0.88599187]

Average Accuracy: 0.9230038320118137

Accuracy after second round - Device Accuracies: [0.94629545 0.89918208 0.95656168 0.94726801 0.88560325]

Final Output

Average Accuracy: 0.9269820918343662

Recall: 0.7599999999848

Precision: 0.7900207900043654

F1-Score: 0.7747196687883238

Accuracy: 0.9598181818164367

7. Novel work

Co-Teaching

- Two MLP neural network models are trained simultaneously.
- Each model selects low-loss (reliable) samples during training.
- The selected clean samples are exchanged between both models.
- Noisy or incorrect samples are gradually ignored using a forget-rate strategy.
- This improves robustness, reduces overfitting, and enhances generalization.
- Co-teaching helps improve intrusion detection performance on noisy and imbalanced datasets.

Advantages

- Better handling of noisy labels
- Improved attack detection capability
- Higher recall and model stability
- More generalized learning compared to single-model training

Output -

Recall: 0.93999999999812

Precision: 0.6216931216848982

F1-Score: 0.7484076385079009

Accuracy: 0.9425454545437408

8. Advantages of RePAIR-Based IDS

- Handles noisy labels effectively.
- Improves minority attack detection.
- Enhances model generalization.
- Reduces overfitting.
- Improves robustness against noisy traffic patterns.
- Better intrusion classification performance.

9. Limitations

- Training process is computationally expensive.
- GAN training requires more resources and tuning.
- Deep learning models require large datasets.
- Ensemble methods increase training complexity.

10. Future Work

- Use larger real-time intrusion datasets.
- Apply advanced transformer-based models.
- Improve real-time attack detection capability.
- Integrate federated learning for privacy preservation.
- Optimize training complexity and inference speed.

11. References

- 1CICIDS Dataset - [Network Intrusion Dataset](#)
- Research Paper - [Research Paper](#)
- Github code - [code](#)